QUEEN'S UNIVERSITY QUIZ 2 VERSION C
FACULTY OF ARTS AND SCIENCE
SCHOOL OF COMPUTING

CISC 220 001 – Professor Burton Ma
November 14, 2023

INSTRUCTIONS TO STUDENTS:
This quiz is 50 MINUTES in length.
There are 4 questions worth a total of 30 marks in this examination.
Please answer all questions in the space provided. There is an extra page that you may use.
No aids are allowed.

Put your name and student number in the space indicated below.

GOOD LUCK!

PLEASE NOTE:
Asking questions during the test is discouraged.
Do your best to answer quiz questions as written.

This material is copyrighted and is for the sole use of students registered in CISC 220 and writing this quiz. This material shall not be distributed or disseminated. Failure to abide by these conditions is a breach of copyright and may also constitute a breach of academic integrity under the University Senate's Academic Integrity Policy Statement.

NAME:

STUDENT NUMBER:

Question 1 (1 mark each, 12 marks total):

Your answer for each of the following parts should be very brief.

A) What does the metacharacter `?` mean in a regular expression?

zero or one instances of the preceding atom

B) Provide a regular expression that matches all strings that start and end with a lowercase English letter.

```
^[a-z]+$          correct answer
^[a-z]*[a-z]$     correct answer
^[a-z].*[a-z]$    sufficient for quiz (does not match single char)
```

C) Show how to create a Bash array `arr` that contains the values 1, 2, and 3.

```
arr=(1 2 3)
```

D) Given a Bash array `arr`, show how to get the number of elements in `arr`.

```
${#arr[*]}    ${#arr[@]}
```

E) Suppose that you have a variable `f` that stores the name of a file. Write a Bash `if` statement that prints the message `not readable` to standard output if the file whose name is stored in `f` is not a readable file.

```
if [[ ! -r $f ]]; then echo “not readable”; fi
```

F) Fill in the blank in the following Bash for loop so that the loop processes each command line argument given to a script:

```
for ____arg in ”$@”____________________; do
```

```
    # loop body not shown
done
```

G) What does the C preprocessor do when it encounters an `#include` directive in a C source code file?

Inserts the included file into the source code file.

H) Every executable program in Linux sets an exit status when the program completes. How does a C program set its exit status?

The return value of the main function determines the exit status.

I) Show how to use the C function `printf` to print an `int` value stored in a variable named `x`:

```
printf("%d", x);
```

J) In C, what is the lifetime of an object having static storage duration?

The lifetime of the program.

K) What is the value of `sizeof(char)` in C?

1

L) What character is used to indicate the end of a C string?

`'\0'`

Question 2 (2 marks each, 4 marks total):

A) Suppose you have an array `arr` of `int` values. Show how to create a pointer `p` that points at the third element of `arr`.

```
int *p = arr + 2;
int *q = &arr[2];
```

B) Consider the following C function that searches an array `arr` of `n` positive values for the minimum value in the array and returns the index of the minimum value; the array is assumed to have at least one value:

```
int minidx(int n, int arr[n]) {
    int index = 0;
    int lo = arr[index];
    for (int i = 1; i < n; i++) {
        if (arr[i] < lo) {
            index = i;
            lo = arr[i];
        }
    }
    return index;    // your answer should change only this line!
}
```

Instead of returning an index, how would you change the `return` statement so that the function returned a pointer to the minimum value instead of returning an index (assume that the return type is changed to `int *`)? Your answer should change only the `return` statement; do not modify any other part of the function.

```
return arr + index;   // return &arr[index];
```

Question 3 (4 marks):

Complete the following C program shown below by completing the `negate` function and creating a `main` function that calls `negate`:

```
void negate(int *p, int *neg) {
    // Gets the int value pointed at by p, computes and stores the
    // negative of the int value in the object pointed at by neg
    //
    // For example, if p points at an int whose value is 5, then
    // after calling the function, the value -5 is stored in
    // the int object pointed at by neg
    *neg = -(*p);



}

// add a main function that correctly calls negate
// (you pick the value of the int object that x points at)

int main(void) {
    int x = 1;
    int y;
    negate(&x, &y);
    return 0;
}
```

Question 4 (10 marks):

The selection sort algorithm for sorting an array `arr` of *n* integers can be described as follows:

for `j`=0, 1, 2, ..., *n*-1
  `idx = minidx(j)`
  swap `arr[j]` and `arr[idx]`

The function call `minidx(j)` finds the index of the minimum element in `arr` starting at index `j` and going to the end of `arr`. For example, if `arr` is the array

`[0, 3, 1, 4, 2]`

then

`minidx(0)` returns `0`
`minidx(1)` returns `2`
`minidx(2)` returns `2`
`minidx(3)` returns `4`
`minidx(4)` returns `4`

Complete the following Bash script so that it implements the selection sort algorithm to sort the command line arguments provided to the script. The functions `sort` and `minidx` are the selection sort algorithm and minimum index algorithm described above.

```
#!/bin/bash

function sort {
}

function minidx {
}

# places the command line arguments into arr and calls sort
arr=(“$@”)
sort

# print the sorted elements of arr on a single line after this comment
```

(instructions continue on next page)

You may answer this question by providing implementations of the `sort` and `minidx` functions and by showing how to print the elements of the sorted array on a single line.

No error checking is required. You may assume that the command line arguments provided to the script are always integers.

You may choose how to call `minidx` and how to return a value from `minidx`.

```
function sort {
    for (( j = 0; j < n; j++ )); do
        minidx $j
        local tmp=${arr[j]}
        arr[j]=${arr[idx]}
        arr[idx]=$tmp
    done
}

function minidx {
    local start=$1
    local lo=${arr[$start]}
    idx=$start
    for (( i = start + 1; i < n; i++)); do
        elem=${arr[i]}
        if (( $elem < $lo )); then
            lo=$elem
            idx=$i
        fi
    done
}

# number of elements
n=${#arr[@]}

# to print the elements
echo “${arr[@]}”
```